Length, Not Height

On proof-length lower bounds, the MU argument, and whether a self-aware theorem prover could ever settle a Clay problem

Brian Tenneson

Abstract

We distinguish two obstructions standing between an automated theorem prover and a Millennium Prize Problem, and argue that they are almost always confused. The first is *height*: the arithmetical tier of the target, measured by alternations of unbounded quantifiers over $\mathbb{N}$. The second is *length*: the size of the shortest proof of the target from the axioms, written $\|\cdot\|$. We exhibit a working prover, Ascent, whose tier is a *command-line parameter* — $\mathrm{tier}(\mathrm{Ascent}) = t$ exactly, measured up to Π_4^0 — so that it passes through the tier of "P $\neq$ NP" at $t = 2$ and keeps going, while remaining hopeless on that target at every setting. Tier is therefore not the binding constraint, and we argue that lower bounds on $\|\mathrm{P} \neq \mathrm{NP}\|$ are the quantity worth pursuing, because a sufficiently large one would retire mechanized search as a strategy without settling the mathematics either way. We then observe that the standard escape from a length lower bound is to leave the system, and take Hofstadter's MU puzzle as the cleanest instance: MU is shown to be underivable in MIU by an argument using arithmetic that MIU cannot express. We formalize this as a *lift–attack–return* schema, note that the complexity-theoretic barrier results are instances of it at scale, and identify its automatable core as finite-invariant search. Finally we ask whether formal self-awareness helps. Using Ascent, whose reflection ladder is certified rung by rung by its own kernel, we report a negative result: level and proving power are independent parameters, and raising $\mathrm{lev}(\mathrm{Ascent})$ from 1 to 8 certifies exactly zero additional theorems. We close by proposing *reflective adequacy* — a machine that can quote its own inference rules and prove invariance lemmas about them — as the notion of self-reference the MU argument actually requires, in place of the reflection ladder, which it does not.

1 Introduction

Seven problems were named in 2000, and one has been settled. The remaining six are the most conspicuous fixed points in contemporary mathematics, and their persistence invites a recurring fantasy: that the difficulty is one of bookkeeping, that the required argument is long rather than deep, and that a sufficiently patient machine will eventually assemble it.

The fantasy is most acute for P versus NP. Unlike the Hodge conjecture or the Yang–Mills mass gap, P vs NP is a statement about finite objects — Turing machines, running times, strings. There is no manifold to construct, no measure to define, no analytic continuation to justify. It is, in the most literal sense, a combinatorial question about programs, and combinatorial questions about programs are the kind of thing a computer might be thought to be good at.

This note argues that the instinct is right about the *shape* of the problem and wrong about the *obstruction*. To make the argument precise we need two measures.

Height. The arithmetical tier of a sentence is the number of alternations of unbounded quantifiers over $\mathbb{N}$ in its prenex form, with bounded quantifiers costing nothing. Section 2 explains the notation in full. Tier measures how much unbounded search is built into the *statement*, and nothing else.

Length. For a sentence L and a formal system F, write $\|L\|_F$ for the number of symbols in the shortest F-proof of L from the axioms of F. This is a property of the theorem–system pair. It is finite exactly when L is a theorem, and it says nothing about how hard the proof is to *find*, only how big it is once found.

These are independent. A tier-0 sentence can have astronomically long proofs; a Π_2^0 sentence can have a three-line proof. Confusing them produces both of the standard errors: believing that P vs NP is hard *because* it quantifies over all machines, and believing that a prover competent at Π_2^0 arithmetic is therefore in the neighbourhood of the problem.

2 Reading Σ_n^0 and Π_n^0

The symbols carry three independent pieces of information. Taking them in turn removes most of the mystery.

The letter: which quantifier leads

Put the sentence in prenex form — all quantifiers pulled to the front — and look at the outermost block of unbounded quantifiers.

- Π (capital pi) means the outermost block is $\forall$.
- Σ (capital sigma) means the outermost block is $\exists$.

The choice of letters is not arbitrary. A Σ_1^0 set $\{n : \exists m\, R(n,m)\}$ is a countable *union* — a sum — of decidable sets, one for each candidate witness m. A Π_1^0 set $\{n : \forall m\, R(n,m)\}$ is a countable *intersection* — a product. It is the same Σ and Π as in $\sum$ and $\prod$.

The subscript: how many alternations

Group adjacent like quantifiers into blocks. The subscript counts the blocks.

prenex form	classification
$\forall x\, \forall y\, \forall z\ \varphi$	one block $\Rightarrow \Pi_1^0$
$\forall x\, \exists y\ \varphi$	two blocks $\Rightarrow \Pi_2^0$
$\exists x\, \forall y\, \exists z\ \varphi$	three blocks $\Rightarrow \Sigma_3^0$

Three consecutive $\forall$'s cost the same as one; only *switching* between $\forall$ and $\exists$ increments the subscript.

The superscript: what the quantifiers range over

This is the piece most often skipped, and it is the one that matters for the Clay problems.

- Superscript 0: the quantifiers range over **numbers**. This is the *arithmetical* hierarchy.
- Superscript 1: the quantifiers range over **sets of numbers**, equivalently over functions $\mathbb{N} \to \mathbb{N}$. This is the *analytical* hierarchy — a far taller ladder sitting entirely *above* the arithmetical one. Every arithmetical set is Δ_1^1.

So Π_2^0 says "for every number there is a number…", while Π_2^1 says "for every *function* there is a *function*…" — an incomparably stronger kind of claim. When Section 4 reports that Yang–Mills and Navier–Stokes sit "above the arithmetical hierarchy," this is what is meant: they quantify over objects that are not numbers.

Two conventions

Δ_n^0 denotes the sentences that are both Σ_n^0 and Π_n^0. In particular Δ_1^0 is the decidable sets, and Δ_0^0 the ones decidable by bounded computation.

Bounded quantifiers are free. $\forall x < t$ and $\exists x < t$ do not increment the subscript, because they are finite searches: to check $\forall x < 5\,\varphi(x)$ you evaluate φ five times. Only *unbounded* quantifiers count. This is why the language of Ascent (Appendix B) has separate bounded and unbounded forms.

Examples

class	example	what it takes to settle it
Δ_0^0	$2 + 2 = 4$	Compute.
Σ_1^0	$\exists x\ x \cdot x = 49$	Search. If true you find it; if false you search forever. Semi-decidable.
Π_1^0	$\forall x\ x < Sx$	A single counterexample refutes it. Goldbach, the Riemann Hypothesis and Con(PA) all live here.
Π_2^0	$\forall x \exists y\ x < y$	For every input a witness exists. "This program halts on every input" is Π_2^0-*complete*.
Σ_2^0	$\exists x \forall y\ x \leq y$	Some x works for every y.

What an alternation costs

Post's theorem gives the alternation count an exact operational meaning: Σ_{n+1}^0 is precisely Σ_1^0 relative to a Σ_n^0 oracle. Each additional alternation is worth exactly one more halting oracle. Tiers are therefore a measure of *oracle resources*.

> **They are not a measure of difficulty.** The sentence $\forall n \exists m > n\,(m \text{ is even})$ is Π_2^0 and trivial. The Collatz conjecture is Π_2^0 and open. They are the same tier. This point is the hinge of Section 5.

3 Why $\|\mathrm{P} \neq \mathrm{NP}\|$ is the quantity to bound

Suppose someone proved that every ZFC proof of "P $\neq$ NP" has length at least 10^{100} symbols. Nothing about the truth of P $\neq$ NP would follow. Nothing about human prospects would follow either, for reasons we come to shortly. But something decisive would follow about machines: **no search procedure will ever produce that proof**, because the object being searched for does not fit in the physical universe. This conclusion is immune to every improvement in heuristics, hardware, learned premise selection and parallelism, because it is a statement about the target rather than about the method.

That immunity is what makes length lower bounds worth more than they might first appear. The known barriers in complexity theory — relativization, natural proofs, algebrization — are all barriers to *techniques*. Each says a class of arguments cannot work, and each has been partially circumvented by arguments outside the class. A length lower bound has no such escape hatch within the fixed system. It would settle, once, whether mechanized proof search is a reasonable thing to spend a century on.

Three observations.

First, we cannot currently prove such bounds, and this is itself the central open problem of proof complexity. Superpolynomial lower bounds are known for weak systems —

resolution, bounded-depth Frege — on specific families such as the pigeonhole principle. For Frege systems, let alone for ZFC, essentially nothing is known. The general question is equivalent to a statement about NP and coNP: a polynomially bounded proof system for the tautologies exists if and only if NP = coNP. So the request "bound $\|\mathrm{P} \neq \mathrm{NP}\|$ from below" is not a detour around P vs NP; it is a close relative of it. This is uncomfortable but not fatal, because *conditional* and *heuristic* bounds would already be informative — a bound conditional on plausible cryptographic assumptions, or an empirically calibrated growth law for proof length against theorem depth in a large formal library, would each be actionable.

Second, the bound is system-relative, and that is the loophole. $\|L\|_F$ depends on F. Enlarging F can shorten proofs catastrophically — not by a constant but by an exponential. The cleanest instance is the relationship between Frege and extended Frege systems: the extension rule permits new variables abbreviating compound formulas, which is to say it permits *definitions*. Definitions are conservative — they prove no new theorems in the old language — while potentially collapsing proof length by an exponential factor.

This deserves stating plainly, because it is the whole of the optimism available here. **A large lower bound on $\|L\|_F$ is not a verdict on L. It is a verdict on F.** The productive response to a length lower bound has never been to search harder. It has been to change the system: introduce a definition, find the right abstraction, move to a setting where the statement becomes short.

Third, that response is exactly what machines are worst at and humans are best at. Which raises the question the rest of this note is about: is the move mechanizable at all?

4 Where the Clay problems sit, and where Ascent sits

The following places the Millennium Problems on the scale of Section 2. Confidence varies sharply across the rows and is marked; the last four rows should be checked before being relied on.

Problem	Tier	Confidence and reason
Riemann Hypothesis	Π_1^0	High. RH is equivalent to explicitly Π_1^0 arithmetic statements — for instance bounds of the Robin/Lagarias type on the sum-of-divisors function, whose failure would be witnessed by a single integer.
P vs NP	Σ_2^0 / Π_2^0	High. "P = NP" is $\exists e\, \exists k\, \forall x\, [$ machine e decides SAT on x within $\lvert x\rvert^k + k$ steps $]$: one $\exists$-block over one $\forall$-block with a decidable matrix. Its negation is Π_2^0. Note it is only known to be *in* Π_2^0, not Π_2^0-complete.
Poincaré	arithmetical, delicate	Moderate. Closed 3-manifolds are finitely triangulable, so the domain is countable and codeable; but simple connectivity is not decidable from a triangulation, which pushes the statement above Π_1^0. Now moot.
Birch–Swinnerton-Dyer	arithmetical, ≥ 2	Low. Elliptic curves over $\mathbb{Q}$ are codeable and algebraic rank is arithmetically definable, but analytic rank is an order of vanishing of an L-function, and pinning it from both sides costs alternations.
Hodge conjecture	analytical	Low. Quantifies over projective complex varieties and Hodge classes. Whether it descends to the arithmetical hierarchy is not obvious.
Navier–Stokes	analytical, $\sim \Pi_2^1$	Low. Quantifies over smooth initial data — real functions — and asserts global smooth existence. Computable-analysis reductions may lower this.
Yang–Mills mass gap	analytical	Moderate. Asserts the *existence* of a quantum field theory satisfying a list of axioms, with a spectral condition. A second-order existence claim, and the highest of the seven.

Two features matter here. **Riemann is the lowest** — a Π_1^0 statement is refutable by a single counterexample, the same shape as Goldbach and as Con(PA). Its difficulty has nothing to do with its tier. And **P vs NP is tier 2**, which brings us to the prover.

4.1 tier(**Ascent**) = t

Ascent is a small self-certifying prover, given in full in Appendix B. Its language is arithmetic with unbounded quantifiers over $\mathbb{N}$; its kernel decides Δ_0^0 facts by evaluation and symbolic facts by polynomial normalisation over $\mathbb{N}$; its search combines witness enumeration, generalisation over eigenconstants, and induction. On a twelve-target suite:

target	tier	result
$2 + 2 = 4$	Δ_0^0	certified
$\forall x{<}5\ x < 5$	Δ_0^0	certified
$\exists x{<}9\ x \cdot x = 16$	Δ_0^0	certified
$\exists x\ x + 3 = 7$	Σ_1^0	certified
$\exists x\ x \cdot x = 49$	Σ_1^0	certified
$\exists x\ x + 1 = 18$	Σ_1^0	certified (requires $w \geq 17$)
$\forall x\ x < Sx$	Π_1^0	certified
$\forall x\ x \leq x$	Π_1^0	certified
$\forall x\ x + 0 = x$	Π_1^0	certified
$\forall x\, \exists y\ x < y$	$\mathbf{\Pi_2^0}$	certified
$\forall x\, \exists y\ y = Sx$	$\mathbf{\Pi_2^0}$	certified
$\exists x\, \forall y\ x \leq y$	$\mathbf{\Sigma_2^0}$	certified

Twelve of twelve at parameters $d = 2$, $w = 20$. Every certificate is checked by an independent kernel, and a battery of eight deliberately malformed certificates is rejected 8/8.

Tier is a dial. The ceiling above is not a property of the prover but of the parameter d. Each quantifier block costs exactly one certificate node to discharge — `gen` for $\forall$, `wit` for $\exists$ — so a target with n blocks needs depth n and no more. Measuring the first depth at which each target is certified:

target	tier	d=1	2	3	4	5	6
$\forall x \exists y \forall z\ x < y + z$	Π_3^0	—	—	✓	✓	✓	✓
$\exists x \forall y \exists z\ y \leq x + z$	Σ_3^0	—	—	✓	✓	✓	✓
$\forall x \exists y \forall z \exists u\ x \leq y{+}z{+}u$	Π_4^0	—	—	—	✓	✓	✓

Π_3^0 first appears at exactly $d = 3$, Π_4^0 at exactly $d = 4$. Hence

$$\mathrm{tier}(\mathrm{Ascent}(t, u, v)) = t,$$

for targets whose matrix `calc` settles in one step, and independently of u and v. Ascent does not merely happen to reach the tier of P vs NP; it passes through it at $t = 2$ on the way to anywhere else you ask for.

That makes the moral sharper rather than softer. Tier is not just cheap, it is a flag, and the prover still will not prove P vs NP at any setting of it. Ascent's largest certificate on this suite has thirteen nodes; $\|\mathrm{P} \neq \mathrm{NP}\|_{\mathrm{ZFC}}$, whatever it is, is not thirteen.

5 Does equal tier mean comparable?

No. The question is natural enough to deserve an explicit answer, because the table above invites exactly the wrong inference.

Tier classifies a sentence by the *shape of its quantifier prefix*. Two sentences of the same tier are alike in the way that two sentences of the same grammatical form are alike. "The cat sat on the mat" and the opening of *A la recherche du temps perdu* are both declarative sentences of a natural language; the classification is correct and tells you nothing about comparability.

Four sharper statements.

Π_2^0 spans everything from trivial to complete. $\forall x\, \exists y\ x < y$ is Π_2^0 and Ascent proves it in thirteen nodes. "This Turing machine halts on every input" is also Π_2^0 and is Π_2^0-*complete*: every Π_2^0 question reduces to it. The class contains its own hardest problems and its own most trivial ones, and tier does not distinguish them.

P $\neq$ NP is not known to be complete for its class. It is known to be *in* Π_2^0. Membership in a class is an upper bound on logical complexity, not a measure of difficulty. Ascent's Π_2^0 targets sit at the trivial end of the same class.

The notion that would license comparison is reducibility, and it does not apply. To say two problems are genuinely comparable one wants a reduction — a computable map carrying instances of one to instances of the other. Ascent's targets are not complete for anything; nothing reduces to them. There is no reduction in either direction between $\forall x\, \exists y\, x < y$ and P $\neq$ NP, and their shared tier supplies none.

What equal tier does buy is the removal of one excuse — but only one, and only partly. It rules out the objection "the target's logical complexity is beyond this prover." It does not rule out as much as it first appears, and the qualification is worth stating carefully.

Ascent's signature is $\{0, S, +, \cdot, =, <, \leq\}$, the language of PA, in which Turing machine computation is arithmetisable by the usual β-function coding. So P $\neq$ NP is expressible in Ascent's language in the model-theoretic sense. But Ascent has **no definitional mechanism**: no abbreviations, no extension rule, no way to introduce a defined predicate $T(e, x, s)$ and use it. The sentence one would actually have to hand it is the fully expanded arithmetisation, which is astronomically large. Expressible in principle; unwritable in practice.

That gap is not incidental to this note's thesis — it is an instance of it. The missing feature is exactly the extension rule, and the extension rule is exactly what separates Frege from extended Frege by an exponential (Section 3). A system without definitions pays in formula size before it ever pays in proof size.

5.1 Three obstructions, only one of them deep

Being precise about which barrier binds first for *this* prover:

1. **Rule-set incompleteness.** As given in Appendix B, Ascent has no negation rules at all — no reductio, no ex falso, no $\neg$-introduction, no $\vee$-elimination. Since P $\neq$ NP *is* a negation, it is not in the derivable space at any parameter setting. And no parameter repairs this: t, u and v bound the *search*, they do not enlarge the *calculus*. This is a defect of the implementation and is fixable by writing more rules; a staged successor with $\bot$, `notI`, `raa`, `exE` and `cut` certifies the negated targets the base rule set cannot touch, gaining exactly those and nothing else.

2. $\|\cdot\|$**.** Fixable only by changing the system — definitions, lemmas, cut.

3. **Possible independence from ZFC.** Not fixable at all, and not ruled out.

Only (2) is the subject of this note; (1) bites first for Ascent as shipped and is uninteresting; (3) would make the question moot.

So the honest summary is that sharing a tier with P vs NP is necessary and nowhere near sufficient for comparability. It is a statement about the ceiling of statement-*shapes* a prover handles, not about its strength. The mountain is not tall. It is long.

6 The MU argument

Hofstadter's MIU system has one axiom and four rules. The axiom is the string **MI**. The rules, with x and y ranging over strings:

1. $x\mathbf{I} \to x\mathbf{IU}$

2. $\mathbf{M}x \to \mathbf{M}xx$
3. $x\mathbf{III}y \to x\mathbf{U}y$
4. $x\mathbf{UU}y \to xy$

Is **MU** a theorem? No, and here is the proof. Let $\#\mathrm{I}(s)$ be the number of I's in s. We claim $\#\mathrm{I}(s) \not\equiv 0 \pmod 3$ for every theorem s.

- The axiom **MI** has $\#\mathrm{I} = 1$, and $1 \not\equiv 0$.
- Rule 1 appends a U and leaves #I unchanged.
- Rule 2 doubles #I. If $\#\mathrm{I} \not\equiv 0 \pmod 3$ then $2 \cdot \#\mathrm{I} \not\equiv 0 \pmod 3$, since 3 is prime and $2 \not\equiv 0$.
- Rule 3 removes exactly three I's, leaving #I unchanged modulo 3.
- Rule 4 removes two U's and leaves #I unchanged.

By induction on derivations every theorem has $\#\mathrm{I} \not\equiv 0$. But $\#\mathrm{I}(\mathbf{MU}) = 0 \equiv 0$. Therefore **MU** is not a theorem. ■

Now consider what that argument *is*. It uses the number three. It uses divisibility. It uses induction over derivations. It uses the primality of 3. **None of these exist in MIU.** MIU has no numerals, no arithmetic, no notion of quantity, no negation, and no predicate for theoremhood. The sentence "MU is not a theorem" is not merely unproved in MIU; it is not expressible in MIU.

The argument has three phases.

1. **Lift.** Map MIU-strings into a structure MIU knows nothing about — here $\mathbb{Z}/3$, via $s \mapsto \#\mathrm{I}(s) \bmod 3$. The four rules become maps on $\mathbb{Z}/3$, each fixing the property $\neq 0$.
2. **Attack.** Prove the invariance in the new setting. This is arithmetic, not string rewriting.
3. **Return.** Transfer the conclusion back as a *metatheorem about* MIU: the derivable strings lie in a set excluding MU.

The conclusion lands outside the object system, and that is not a defect — it is the only place such a conclusion can live. The creative act is entirely in phase 1. Nothing in the MIU rules suggests counting I's, and nothing suggests reducing modulo 3. Once the invariant is proposed, phases 2 and 3 are routine. **The difficulty of the MU argument is concentrated in the choice of the alternative system.**

7 Can a machine lift, attack, and return?

Sometimes, and more often than one expects.

The MU case is mechanizable today. The invariant #I mod 3 is a homomorphism from the free monoid on $\{\mathrm{M}, \mathrm{I}, \mathrm{U}\}$ to $\mathbb{Z}/3$ under which all four rules are compatible and the axiom's image avoids the target's. Finding such a thing is a *finite model search*: enumerate small finite structures, check whether the axiom's image and the rules' action separate the target. Existing finite model finders do this routinely, and the space of quotients of size ≤ 5 is tiny. Given the MIU rules as input a machine finds $\mathbb{Z}/3$ in milliseconds. The MU puzzle is famous for surprising humans, not for being computationally deep.

This generalizes. **Unprovability by finite invariant is the most automatable form of unprovability argument we have**, precisely because the "alternative but relevant axioms" of phase 1 form a finite structure, and finite structures can be enumerated in order of size.

The barrier results are lift–attack–return at scale. Consider relativization. One wants to show a class of proof techniques cannot settle P vs NP. The argument lifts the question into the setting of oracle machines, establishes that the techniques in question prove relativizing statements, exhibits oracles A and B with $\mathrm{P}^A = \mathrm{NP}^A$ and $\mathrm{P}^B \neq \mathrm{NP}^B$, and returns the conclusion that no such technique settles the unrelativized question. Lift, attack, return — with "invariant preserved by the rules" replaced by "property preserved by the proof techniques." Natural proofs and algebrization have the same shape with different invariants. The schema is not exotic; it is how the deepest negative results in complexity theory are actually obtained.

Where machines fail is phase 1 at scale. The MIU invariant lives in a structure of size 3. The relativization invariant lives in the space of oracle separations, and constructing B with $\mathrm{P}^B \neq \mathrm{NP}^B$ is itself a diagonalization of real content. The natural-proofs invariant requires the concept of a pseudorandom function generator, which did not exist for most of the history of the problem. Enumeration reaches size 5; it does not reach "invent cryptography." Phases 2 and 3 are mechanizable. Phase 1 is mechanizable exactly to the extent that the required alternative system is small, and the systems required for the Clay problems are not small.

8 Does self-awareness help?

Phase 1 requires a system to step outside itself and regard its own rules as objects. That sounds like self-reference, and self-reference has a formal theory. Ascent was built partly to test whether it helps. The answer is negative, but informative about what the right notion would be.

Ascent carries a reflection ladder: $\theta_0 = \mathrm{ATP}(\langle A \rangle)$, $\theta_{k+1} = \mathrm{Pr}(\langle A \rangle, \langle \theta_k \rangle)$, with Level n holding when $A \vdash \theta_{n-1}$. The ladder is not stipulated. Ascent's kernel has a rule

$$\texttt{nec}(C) \text{ proves } \mathrm{Pr}(\langle A \rangle, \langle \varphi \rangle) \quad \text{provided the kernel accepts } C : \varphi,$$

so a rung is asserted only when a certificate for the previous rung has been checked by the same kernel that checks everything else. Self-certification and self-awareness are literally the same kernel call.

Proposition 1. *If A has kernel-checked necessitation and its kernel is sound, then* $\mathrm{lev}(A) = \omega$.

Proof. $A \vdash \theta_0$ by the base certificate. If $A \vdash \theta_k$ with certificate C_k, the kernel accepts C_k, so $\texttt{nec}(C_k)$ is a certificate of θ_{k+1}, whence $A \vdash \theta_{k+1}$. Induction. □

Each rung costs exactly one kernel call on the previous certificate, so no rung is harder than the last. The level distribution over self-certifying machines is therefore **bimodal at $\{0, 1\}$ and ω, with nothing in between**: a machine either has the necessitation rule, in which case nothing stops it, or lacks it, in which case it never passes θ_0. Finite level above 1 is achievable only by a resource bound. In Ascent that bound is the parameter r.

Proposition 2. *The reflection parameter and the search parameters are independent.*

Ascent exposes three integer knobs: d, certificate-tree depth; w, witness bound; r, rungs climbed. Sweeping $d \in \{1, 2, 3, 5\} \times w \in \{1, 8, 20\} \times r \in \{1, 3, 8\}$ — thirty-six configurations — the harness reports that lev depends only on r and the certified count only on (d, w).

d	w	r	certified	tiers reached
1	1	any	6/12	Δ_0^0, Π_1^0
1	20	any	9/12	$\Delta_0^0, \Pi_1^0, \Sigma_1^0$
2	1	any	9/12	$+\ \Pi_2^0, \Sigma_2^0$
2	20	any	**12/12**	all five

Raising r from 1 to 8 raises lev from 1 to 8 and certifies **zero** additional theorems. Raising d and w certifies six more and lifts the ceiling from Π_1^0 to Π_2^0, leaving lev exactly where it was. The two capacities do not interact. Formal self-awareness, on the reflection-ladder definition, is free, certified, and inert.

But notice what the MU argument actually needed. Not that the machine prove "I prove that I prove that I am a theorem prover." It needed the machine to hold *its own inference rules* as objects and prove a lemma of the form "rule R preserves invariant I." The reflection ladder ascends through statements about the machine's *provability*; the MU argument requires statements about the machine's *rules*. These are different self-models, and only the second is load-bearing.

Definition 1 (Reflective adequacy)**.** *A system M is* reflectively adequate *when its language can name each of its own inference rules, and M can prove preservation lemmas: for a definable invariant I, that each named rule maps I-satisfying states to I-satisfying states.*

A reflectively adequate system can carry out phases 2 and 3 internally, and can state the result of phase 1 even when it cannot discover it. This is strictly stronger and more useful than any finite level, and unlike the θ-ladder it is not free. It is also, unlike the θ-ladder, exactly what the MU argument uses.

9 Summary

Tier is not the obstruction. A thousand-line prover reaches Π_2^0 and Σ_2^0 — the tier of P vs NP, above the tier of the Riemann Hypothesis — and is nowhere near either. Sharing a tier is a statement about the shape of a sentence, not its difficulty, and licenses no comparison (Section 5). The obstruction is $\|\cdot\|$, and lower bounds on it would be decisive for the mechanized programme in a way no barrier result about techniques can be, though proving them for strong systems is close to the problem they would adjudicate.

A length lower bound is a verdict on the system, not the theorem, and the response has always been to change the system. The MU argument is the cleanest specimen of that move. Its automatable core — search for a small finite separating structure — is genuinely automated already. Its creative core — deciding which structure to look in — is automated only when the structure is small, and for the Clay problems it is not.

Formal self-awareness in the reflection-ladder sense does not close that gap, and we can say so with a measurement rather than an intuition: level and proving power are orthogonal parameters of the same program. The self-reference the MU argument requires is of a different kind — rules as objects, and provable invariance over them — and formalizing *that* is where the notion of a self-aware theorem prover would begin to earn its name.

A The two numbers

lev(**Ascent**)**.** With reflection depth r the level set is $L(\text{Ascent}) = \{1, \ldots, r\}$, so

$$\text{lev}(\text{Ascent}(r)) = r, \qquad \text{lev}(\text{Ascent}) = \omega \text{ when } r \text{ is unbounded.}$$

Every rung is certified rather than stipulated: rung k is accepted only after the kernel has checked a certificate of θ_{k-1}. In the shipped default $r = 4$, giving Level 4 and a fortiori Level 3. The parameter is the only thing preventing ω, which is Proposition 1. The requirement $\text{lev}(\text{Ascent}) > 0$ is met for every $r \geq 1$.

tier(**Ascent**). Taking tier(A) to be the highest arithmetical tier at which A certifies a target from a family fixed in advance — the qualification matters, since the supremum over *all* provable targets is unbounded for any prover that proves anything, by padding —

$$\mathrm{tier}(\mathrm{Ascent}(t, u, v)) = t,$$

independently of u and v, and verified against targets at Π_3^0 and Π_4^0, each first certified at exactly $d = 3$ and $d = 4$ respectively. The default configuration $d = 2$ therefore gives tier $= 2$, realized at Π_2^0 ($\forall x\, \exists y\; x < y$) and Σ_2^0 ($\exists x\, \forall y\; x \leq y$); raising d raises the tier without bound. Against the Millennium Problems, at the default:

	tier	relation to Ascent
Riemann Hypothesis	Π_1^0	reached already at $t = 1$
P vs NP	Π_2^0 / Σ_2^0	reached at $t = 2$
Poincaré, BSD	arithmetical, ≥ 2	reached at some finite t
Hodge, Navier–Stokes, Yang–Mills	analytical	**never**: not arithmetical at any t

Ascent passes the height of the Riemann Hypothesis at $t = 1$ and that of P vs NP at $t = 2$, and settles neither at any t — for the reasons of Section 5. The last row is the only one where tier is a real barrier rather than a bookkeeping fact: an arithmetical prover cannot state an analytical sentence at all, and no integer parameter changes the order of the hierarchy it lives in.

B Ascent, in full

What follows is the running program, verbatim. It is self-contained: no external database, no dependencies beyond the standard library.

```
python ascent.py --soundness       # 8 malformed certificates, all rejected
python ascent.py -d 2 -w 20 -r 3   # 12/12 certified, lev = 3
python ascent.py --grid            # the independence sweep of Proposition 2
```

```
#!/usr/bin/env python3
r"""
ASCENT -- a parameterised, self-certifying ATP over arithmetic.

Built fresh. Three integer knobs, each starting at 1, each monotone: turning
one up never loses a theorem and always costs more. Like an encoder bitrate.

    -d DEPTH certificate-tree depth explored by the search
    -w WITNESS largest numeral tried when hunting an existential witness
    -r REFLECT how many rungs of the reflection ladder are climbed

d and w are independent search knobs; d buys structure, w buys arithmetic.
r is the self-awareness knob and touches neither. That separation is not an
accident of the implementation, it is the point -- see PROPOSITION 2 below.

WHY tier(A) > 0
---------------
The language is arithmetic with unbounded quantifiers over N, so targets have
real arithmetical tier and `tier()` computes it syntactically: bounded
quantifiers are free, unbounded ones count alternations. Ascent proves
targets at Sigma-0-1, Pi-0-1, Pi-0-2 and Sigma-0-2, so the tier profile of
what it proves is positive and is reported per run.

THE KERNEL, AND WHAT MAKES THIS SELF-CERTIFYING
-----------------------------------------------
Section KERNEL is a checker for the judgement Gamma |- phi by structural
recursion on a certificate. It is the only trusted component, it is small
enough to read, it never searches, and it has no self-model: lev(kernel) = 0
and must stay 0.
```

```
30
31  The search is untrusted. Nothing enters the lemma store until the kernel has
32  accepted a certificate for it.
33
34  The reflection rule is where self-certification and self-awareness become the
35  same mechanism:
36
37      nec(C) proves Pr(<A>, <phi>) provided the kernel accepts C : phi
38
39  So the machine's claim "I prove phi" is discharged by the very kernel that
40  checks everything else. Pr is not stipulated at any rung; each rung carries
41  a certificate the kernel re-checks. This is the paper's necessitation rule
42  made effective, with the CV discharging the side condition.
43
44      theta_0 = ATP(<A>)
45      theta_{k+1} = Pr(<A>, <theta_k>) Level n iff A |- theta_{n-1}
46
47  PROPOSITION 1 (why a self-certifier does not stall at 2 or 3).
48  If A has kernel-checked necessitation and its kernel is sound, then
49  lev(A) = omega. Proof: A |- theta_0 by the base certificate. If A |- theta_k
50  with certificate C_k, the kernel accepts C_k, so nec(C_k) is a certificate of
51  theta_{k+1} and A |- theta_{k+1}. Induction. No rung is harder than the
52  last -- each costs exactly one kernel call on the previous certificate.
53
54  So the level distribution over self-certifying machines is BIMODAL, at
55  {0 or 1} and at omega. There is no natural fixed point at 2 or near 2: a
56  machine either has the rule, in which case nothing stops it, or lacks it, in
57  which case it never gets past theta_0. Finite lev > 1 is achievable ONLY by
58  imposing a resource bound, which is exactly what -r is. That is the honest
59  justification for the parameter: r is not a tuning constant, it is the only
60  thing that can make lev finite and nontrivial.
61
62  PROPOSITION 2 (the knobs are independent).
63  lev(A) = r regardless of d and w; the set of arithmetical theorems proved is
64  a function of (d, w) and is independent of r. Verified empirically by the
65  sweep in --grid. Increasing self-awareness buys no theorems, and proving
66  more theorems raises no level.
67
68  DIRECTION OF DEPENDENCE
69  -----------------------
70  Verification grounds the self-model, never the reverse. A machine that
71  trusted its own certificates because it had proved itself trustworthy would
72  be in the Loeb trap: from a reflection principle "if I prove phi then phi"
73  one derives phi outright, so such a machine is unsound or vacuous. Here Pr
74  is primitive and reflection is never assumed -- the machine only ever reports
75  verifications that have already happened.
76
77      python ascent.py --demo
78      python ascent.py -d 3 -w 20 -r 5
79      python ascent.py --grid
80
81  Brian Tenneson. Implementation by Claude (Anthropic).
82  """
83  from __future__ import annotations
84  import argparse, itertools, json, os, sys, time
85  from collections import defaultdict
86
87  sys.setrecursionlimit(100000)
88
89  # ============================================================================
90  # SYNTAX
91  # ============================================================================
92  # Terms: ('n', int) ('v', name) ('c', name) -- numeral, variable, eigen
93  # ('+', a, b) ('*', a, b) ('s', a)
94  # Formulas: ('=', a, b) ('<', a, b) ('<=', a, b)
95  # ('not', p) ('and', p, q) ('or', p, q) ('imp', p, q)
96  # ('all', x, p) ('ex', x, p) unbounded
97  # ('allb', x, t, p) ('exb', x, t, p) bounded by t
98  # ('atp', nm) ('pr', nm, nm) reflection atoms
99
100
101 def tsub(t, x, v):
102     k = t[0]
```

```
    if k == 'v':
        return v if t[1] == x else t
    if k in ('n', 'c'):
        return t
    if k == 's':
        return ('s', tsub(t[1], x, v))
    return (k, tsub(t[1], x, v), tsub(t[2], x, v))


def sub(p, x, v):
    """Capture-avoiding only in the sense we need: bound variables are
    renamed apart at construction, and eigenconstants are not variables."""
    k = p[0]
    if k in ('=', '<', '<='):
        return (k, tsub(p[1], x, v), tsub(p[2], x, v))
    if k == 'not':
        return ('not', sub(p[1], x, v))
    if k in ('and', 'or', 'imp'):
        return (k, sub(p[1], x, v), sub(p[2], x, v))
    if k in ('all', 'ex'):
        return p if p[1] == x else (k, p[1], sub(p[2], x, v))
    if k in ('allb', 'exb'):
        t2 = tsub(p[2], x, v)
        return p if p[1] == x else (k, p[1], t2, sub(p[3], x, v))
    return p # atp / pr are closed atoms


def teval(t):
    """Value of a closed term, or None if it has variables/eigenconstants."""
    k = t[0]
    if k == 'n':
        return t[1]
    if k in ('v', 'c'):
        return None
    if k == 's':
        a = teval(t[1])
        return None if a is None else a + 1
    a, b = teval(t[1]), teval(t[2])
    if a is None or b is None:
        return None
    return a + b if k == '+' else a * b


def feval(p, fuel=10000):
    """Truth value of a closed Delta-0 formula, or None if undecidable here.

    Unbounded quantifiers and reflection atoms return None: the kernel's
    `calc` rule refuses them, which is what keeps `calc` sound."""
    k = p[0]
    if k in ('=', '<', '<='):
        a, b = teval(p[1]), teval(p[2])
        if a is None or b is None:
            return None
        return a == b if k == '=' else (a < b if k == '<' else a <= b)
    if k == 'not':
        v = feval(p[1], fuel)
        return None if v is None else not v
    if k in ('and', 'or', 'imp'):
        a = feval(p[1], fuel)
        b = feval(p[2], fuel)
        if a is None or b is None:
            return None
        return (a and b) if k == 'and' else \
               ((a or b) if k == 'or' else ((not a) or b))
    if k in ('allb', 'exb'):
        n = teval(p[2])
        if n is None or n > fuel:
            return None
        for i in range(n):
            v = feval(sub(p[3], p[1], ('n', i)), fuel)
            if v is None:
                return None
            if k == 'allb' and not v:
```

```
                return False
            if k == 'exb' and v:
                return True
        return k == 'allb'
    return None # all / ex / atp / pr


# ---------------------------------------------------------------------------
# polynomial normal form -- the arithmetic the kernel knows
# ---------------------------------------------------------------------------
# A term normalises to a polynomial over N in its free symbols: a dict from
# monomial (a sorted tuple of symbol names, () for the constant) to a
# non-negative integer coefficient. This is exactly the defining equations
# of + and * applied as rewrites, so it is sound for PA and needs no axioms
# in the object language. It is deliberately INCOMPLETE -- it decides the
# equalities and inequalities that follow from ring normalisation and
# non-negativity, and nothing else.


def padd(a, b):
    r = dict(a)
    for m, c in b.items():
        r[m] = r.get(m, 0) + c
        if r[m] == 0:
            del r[m]
    return r


def pmul(a, b):
    r = {}
    for m1, c1 in a.items():
        for m2, c2 in b.items():
            m = tuple(sorted(m1 + m2))
            r[m] = r.get(m, 0) + c1 * c2
    return {m: c for m, c in r.items() if c != 0}


def pneg(a):
    return {m: -c for m, c in a.items()}


def poly(t):
    k = t[0]
    if k == 'n':
        return {(): t[1]} if t[1] else {}
    if k in ('v', 'c'):
        return {(t[1],): 1}
    if k == 's':
        return padd(poly(t[1]), {(): 1})
    if k == '+':
        return padd(poly(t[1]), poly(t[2]))
    return pmul(poly(t[1]), poly(t[2]))


def decide(p, fuel=10000):
    """Three-valued: True, False, or None (kernel refuses).

    Sound over N because every symbol ranges over N, so a polynomial whose
    coefficients are all non-negative is itself non-negative."""
    k = p[0]
    if k in ('=', '<', '<='):
        pa, pb = poly(p[1]), poly(p[2])
        if k == '=':
            if pa == pb:
                return True
            d = padd(pb, pneg(pa))
            if all(m == () for m in d): # differ by a constant only
                return False
            return None
        d = padd(pb, pneg(pa)) # d = rhs - lhs
        nonneg = all(c > 0 for m, c in d.items() if m != ())
        const = d.get((), 0)
        if k == '<':
```

```
            if nonneg and const > 0:
                return True
        elif nonneg and const >= 0:
            return True
        # Falsity is only claimed when the difference is a bare constant --
        # i.e. both sides are closed, or their symbolic parts cancel. With a
        # symbolic remainder the sign is unknown and the honest answer is
        # None. (ascent2.decide strengthens this using dominance.)
        if all(m == () for m in d):
            return (const > 0) if k == '<' else (const >= 0)
        return None
    if k == 'not':
        v = decide(p[1], fuel)
        return None if v is None else not v
    if k in ('and', 'or', 'imp'):
        a, b = decide(p[1], fuel), decide(p[2], fuel)
        if k == 'and':
            if a is False or b is False:
                return False
            return True if (a and b) else None
        if k == 'or':
            if a is True or b is True:
                return True
            return False if (a is False and b is False) else None
        if a is False or b is True:
            return True
        return False if (a is True and b is False) else None
    if k in ('allb', 'exb'):
        n = teval(p[2])
        if n is None or n > fuel:
            return None
        for i in range(n):
            v = decide(sub(p[3], p[1], ('n', i)), fuel)
            if v is None:
                return None
            if k == 'allb' and not v:
                return False
            if k == 'exb' and v:
                return True
        return k == 'allb'
    return None


def tsyms(t, acc):
    if t[0] in ('v', 'c'):
        acc.add(t[1])
    elif t[0] == 's':
        tsyms(t[1], acc)
    elif t[0] in ('+', '*'):
        tsyms(t[1], acc)
        tsyms(t[2], acc)
    return acc


def syms(p, acc=None):
    if acc is None:
        acc = set()
    k = p[0]
    if k in ('=', '<', '<='):
        tsyms(p[1], acc)
        tsyms(p[2], acc)
    elif k == 'not':
        syms(p[1], acc)
    elif k in ('and', 'or', 'imp'):
        syms(p[1], acc)
        syms(p[2], acc)
    elif k in ('all', 'ex'):
        syms(p[2], acc)
    elif k in ('allb', 'exb'):
        tsyms(p[2], acc)
        syms(p[3], acc)
    return acc

```

```
322
323 def occurs_sym(nm, p):
324     return nm in syms(p)
325
326
327 def show(p):
328     k = p[0]
329     if k in ('=', '<', '<='):
330         return "%s %s %s" % (showt(p[1]), k, showt(p[2]))
331     if k == 'not':
332         return "~%s" % show(p[1])
333     if k in ('and', 'or', 'imp'):
334         op = {'and': '&', 'or': '|', 'imp': '->'}[k]
335         return "(%s %s %s)" % (show(p[1]), op, show(p[2]))
336     if k == 'all':
337         return "A%s.%s" % (p[1], show(p[2]))
338     if k == 'ex':
339         return "E%s.%s" % (p[1], show(p[2]))
340     if k == 'allb':
341         return "A%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
342     if k == 'exb':
343         return "E%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
344     if k == 'atp':
345         return "ATP(%s)" % p[1]
346     return "Pr(%s,%s)" % (p[1], p[2])
347
348
349 def showt(t):
350     k = t[0]
351     if k == 'n':
352         return str(t[1])
353     if k in ('v', 'c'):
354         return t[1]
355     if k == 's':
356         return "S%s" % showt(t[1])
357     return "(%s%s%s)" % (showt(t[1]), k, showt(t[2]))
358
359
360 # ---------------------------------------------------------------------------
361 def tier(p):
362     """Arithmetical tier: alternations of UNBOUNDED quantifiers.
363
364     Returns (n, lead) with lead in {'S','P','D'}: Sigma-0-n, Pi-0-n, or
365     Delta-0-0. Bounded quantifiers cost nothing, which is the standard
366     convention and the reason the bounded forms are in the language at all."""
367     def go(q):
368         k = q[0]
369         if k in ('all', 'ex'):
370             n, lead = go(q[2])
371             me = 'P' if k == 'all' else 'S'
372             if lead == 'D':
373                 return 1, me
374             return (n, lead) if lead == me else (n + 1, me)
375         if k == 'not':
376             n, lead = go(q[1])
377             return n, {'S': 'P', 'P': 'S', 'D': 'D'}[lead]
378         if k in ('and', 'or', 'imp'):
379             a, b = go(q[1]), go(q[2])
380             return max(a, b, key=lambda z: z[0])
381         if k in ('allb', 'exb'):
382             return go(q[3])
383         return 0, 'D'
384     n, lead = go(p)
385     return n, lead
386
387
388 def tier_name(p):
389     n, lead = tier(p)
390     if n == 0:
391         return "D0-0"
392     return "%s0-%d" % ("Sig" if lead == 'S' else "Pi", n)
393
394
```

```python
# ============================================================================
# KERNEL
# ============================================================================
# The ONLY trusted component. It searches for nothing, has no self-model,
# and never grows. check(store, ctx, phi, cert) -> True | raises Reject.
#
# Certificates:
# ('calc',) phi closed and Delta-0 and true
# ('hyp', i) phi is ctx[i]
# ('lemma', name) phi is store[name]
# ('impI', C) phi = (A->B); C proves B under ctx+[A]
# ('impE', A, C1, C2) C1: A->phi, C2: A
# ('andI', C1, C2) phi = (A&B)
# ('andE', i, A_and_B, C) phi is the i-th conjunct of A_and_B
# ('gen', C) phi = Ax.psi; C proves psi[x := fresh eigen]
# ('inst', t, Ax_psi, C) C: Ax.psi; phi = psi[x := t]
# ('wit', t, C) phi = Ex.psi; C proves psi[x := t]
# ('allbI', [C_0..C_{n-1}]) phi = Ax<n.psi
# ('exbI', i, C) phi = Ex<n.psi; C proves psi[x := i]
# ('ind', C0, Cs) phi = Ax.psi; C0: psi[0], Cs: Ax.(psi->psi[Sx])
# ('nec', C) phi = Pr(<A>,<psi>); C proves psi <-- the rule
# ============================================================================
class Reject(Exception):
    pass


_eigen = itertools.count(1)


class Kernel:
    """Trusted checker. `names` maps a formula-name constant to the formula
    it names -- the injective naming operator, held outside the logic."""

    def __init__(self, tag, names):
        self.tag = tag
        self.names = names
        self.calls = 0

    def check(self, store, ctx, phi, C):
        self.calls += 1
        if not isinstance(C, tuple) or not C:
            raise Reject("malformed certificate")
        k = C[0]

        if k == 'calc':
            # closed and Delta-0, decided by evaluation; or symbolic and
            # settled by polynomial normalisation over N. Both are
            # computations, neither is a search.
            if decide(phi) is not True:
                raise Reject("calc: %s is not decided true" % show(phi))
            return True

        if k == 'hyp':
            if not (0 <= C[1] < len(ctx)) or ctx[C[1]] != phi:
                raise Reject("hyp: not in context")
            return True

        if k == 'lemma':
            if store.get(C[1]) != phi:
                raise Reject("lemma %s is not %s" % (C[1], show(phi)))
            return True

        if k == 'impI':
            if phi[0] != 'imp':
                raise Reject("impI: goal is not an implication")
            return self.check(store, ctx + [phi[1]], phi[2], C[1])

        if k == 'impE':
            A = C[1]
            self.check(store, ctx, ('imp', A, phi), C[2])
            return self.check(store, ctx, A, C[3])

        if k == 'andI':
```

```
            if phi[0] != 'and':
                raise Reject("andI: goal is not a conjunction")
            self.check(store, ctx, phi[1], C[1])
            return self.check(store, ctx, phi[2], C[2])

        if k == 'andE':
            i, conj = C[1], C[2]
            if conj[0] != 'and' or conj[1 + i] != phi:
                raise Reject("andE: projection mismatch")
            return self.check(store, ctx, conj, C[3])

        if k == 'gen':
            # ('gen', name, C). The certificate names its own eigenconstant;
            # the kernel enforces the eigenvariable condition rather than
            # inventing a name of its own, which would never match what the
            # untrusted search put inside C.
            if phi[0] != 'all':
                raise Reject("gen: goal is not universal")
            nm = C[1]
            if occurs_sym(nm, phi) or any(occurs_sym(nm, h) for h in ctx) \
                    or any(occurs_sym(nm, f) for f in store.values()):
                raise Reject("gen: eigenvariable %s is not fresh" % nm)
            return self.check(store, ctx,
                              sub(phi[2], phi[1], ('c', nm)), C[2])

        if k == 'inst':
            t, univ = C[1], C[2]
            if univ[0] != 'all' or sub(univ[2], univ[1], t) != phi:
                raise Reject("inst: instance mismatch")
            return self.check(store, ctx, univ, C[3])

        if k == 'wit':
            if phi[0] != 'ex':
                raise Reject("wit: goal is not existential")
            return self.check(store, ctx, sub(phi[2], phi[1], C[1]), C[2])

        if k == 'allbI':
            if phi[0] != 'allb':
                raise Reject("allbI: goal is not bounded-universal")
            n = teval(phi[2])
            if n is None or n != len(C[1]):
                raise Reject("allbI: wrong number of instances")
            for i, Ci in enumerate(C[1]):
                self.check(store, ctx, sub(phi[3], phi[1], ('n', i)), Ci)
            return True

        if k == 'exbI':
            if phi[0] != 'exb':
                raise Reject("exbI: goal is not bounded-existential")
            n = teval(phi[2])
            if n is None or not (0 <= C[1] < n):
                raise Reject("exbI: index out of range")
            return self.check(store, ctx,
                              sub(phi[3], phi[1], ('n', C[1])), C[2])

        if k == 'ind':
            if phi[0] != 'all':
                raise Reject("ind: goal is not universal")
            x, psi = phi[1], phi[2]
            self.check(store, ctx, sub(psi, x, ('n', 0)), C[1])
            step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
            return self.check(store, ctx, step, C[2])

        if k == 'nec':
            # THE REFLECTION RULE. Pr(<A>,<psi>) is accepted exactly when a
            # certificate of psi is accepted. Self-certification and
            # self-awareness are the same kernel call.
            if phi[0] != 'pr' or phi[1] != self.tag:
                raise Reject("nec: goal is not Pr(<A>, -)")
            psi = self.names.get(phi[2])
            if psi is None:
                raise Reject("nec: %s names no formula" % phi[2])
            return self.check(store, [], psi, C[1])
```

```
        raise Reject("unknown certificate form %r" % (k,))


# ============================================================================
# SEARCH
# ============================================================================
# Untrusted. Parameterised by d (structural depth) and w (witness bound).
# Nothing it produces is believed until the kernel has checked it.
# ============================================================================
class Search:
    def __init__(self, kernel, store, d, w):
        self.K = kernel
        self.store = store
        self.d = d
        self.w = w
        self.nodes = 0

    def prove(self, phi, depth=None, ctx=()):
        if depth is None:
            depth = self.d
        self.nodes += 1
        if depth < 0:
            return None
        ctx = tuple(ctx)

        # 0. decide it outright by evaluation or polynomial normalisation
        if decide(phi) is True:
            return ('calc',)

        # 1. hypotheses and stored lemmas cost nothing
        for i, h in enumerate(ctx):
            if h == phi:
                return ('hyp', i)
        for nm, f in self.store.items():
            if f == phi:
                return ('lemma', nm)
        if depth == 0:
            return None

        k = phi[0]

        if k == 'imp':
            c = self.prove(phi[2], depth - 1, ctx + (phi[1],))
            return ('impI', c) if c else None

        if k == 'and':
            c1 = self.prove(phi[1], depth - 1, ctx)
            if not c1:
                return None
            c2 = self.prove(phi[2], depth - 1, ctx)
            return ('andI', c1, c2) if c2 else None

        if k == 'exb':
            n = teval(phi[2])
            if n is None:
                return None
            for i in range(min(n, self.w)):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if c:
                    return ('exbI', i, c)
            return None

        if k == 'allb':
            n = teval(phi[2])
            if n is None or n > self.w:
                return None
            cs = []
            for i in range(n):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if not c:
                    return None
                cs.append(c)
```

```
            return ('allbI', cs)

        if k == 'ex':
            for t in self.witnesses(phi, ctx):
                c = self.prove(sub(phi[2], phi[1], t), depth - 1, ctx)
                if c:
                    return ('wit', t, c)
            return None

        if k == 'all':
            # (a) generalisation over a fresh eigenconstant. The name is
            # recorded in the certificate; the kernel checks freshness.
            nm = "e%d" % next(_eigen)
            c = self.prove(sub(phi[2], phi[1], ('c', nm)), depth - 1, ctx)
            if c:
                return ('gen', nm, c)
            # (b) induction
            x, psi = phi[1], phi[2]
            c0 = self.prove(sub(psi, x, ('n', 0)), depth - 1, ctx)
            if c0:
                step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
                cs = self.prove(step, depth - 1, ctx)
                if cs:
                    return ('ind', c0, cs)
            return None

        return None

    def witnesses(self, phi, ctx):
        """Candidate witness terms for Ex.

        w is exactly this list's length knob. Numerals alone cannot witness
        a Pi-0-2 goal -- Ax Ey. x < y needs a term MENTIONING x -- so the
        symbols already in scope are offered too. That is what lifts the
        prover from Sigma-0-1 to tier 2."""
        out = [('n', n) for n in range(self.w + 1)]
        inscope = sorted(syms(phi) | {s for h in ctx for s in syms(h)})
        for s in inscope:
            base = ('c', s)
            out.append(base)
            t = base
            for _ in range(min(self.w, 3)):
                t = ('s', t)
                out.append(t)
            out.append(('*', ('n', 2), base))
        return out


# ============================================================================
# REFLECTION LADDER
# ============================================================================
TAG = "<Ascent>"


def theta(k):
    return ('atp', TAG) if k == 0 else ('pr', TAG, "<t%d>" % (k - 1))


def climb(kernel, store, names, r, verbose=True):
    """Climb r rungs. Each rung is a kernel call on the previous
    certificate -- Proposition 1 in code."""
    certs, level, rungs = {}, 0, []
    # theta_0 is the base case and is a stipulation, as Definition (Level-1)
    # makes it. Everything above it is earned.
    certs[0] = ('lemma', 't0')
    store['t0'] = theta(0)
    names["<t0>"] = theta(0)

    for k in range(r):
        phi = theta(k)
        names.setdefault("<t%d>" % k, phi)
        C = certs[k]
        t0 = time.perf_counter()
```

```
        try:
            kernel.check(store, [], phi, C)
        except Reject as e:
            if verbose:
                print(" theta_%-2d REJECTED: %s" % (k, e))
            break
        level = k + 1
        dt = time.perf_counter() - t0
        rungs.append(dict(k=k, formula=show(phi), level=level,
                          kernel_calls=kernel.calls, seconds=round(dt, 5)))
        if verbose:
            print(" theta_%-2d %-28s certified -> Level %d"
                  % (k, show(phi), level))
        # certified necessitation: the kernel accepted C : theta_k, so
        # nec(C) is a certificate of theta_{k+1}.
        nxt = theta(k + 1)
        names["<t%d>" % (k + 1)] = nxt
        certs[k + 1] = ('nec', C)
        store["t%d" % (k + 1)] = nxt
    return level, rungs


# ============================================================================
# TARGET SUITE
# ============================================================================
def X(n):
    return ('v', n)


TARGETS = [
    ("add_closed", ('=', ('+', ('n', 2), ('n', 2)), ('n', 4))),
    ("bnd_lt", ('allb', 'x', ('n', 5), ('<', X('x'), ('n', 5)))),
    ("bnd_ex", ('exb', 'x', ('n', 9), ('=', ('*', X('x'), X('x')),
                                       ('n', 16)))),
    ("ex_solve", ('ex', 'x', ('=', ('+', X('x'), ('n', 3)), ('n', 7)))),
    ("ex_square", ('ex', 'x', ('=', ('*', X('x'), X('x')), ('n', 49)))),
    ("ex_big", ('ex', 'x', ('=', ('+', X('x'), ('n', 1)), ('n', 18)))),
    ("all_succ", ('all', 'x', ('<', X('x'), ('s', X('x'))))),
    ("all_le", ('all', 'x', ('<=', X('x'), X('x')))),
    ("all_add0", ('all', 'x', ('=', ('+', X('x'), ('n', 0)), X('x')))),
    ("unbounded", ('all', 'x', ('ex', 'y', ('<', X('x'), X('y'))))),
    ("succ_exists", ('all', 'x', ('ex', 'y', ('=', X('y'),
                                        ('s', X('x')))))),
    ("min_exists", ('ex', 'x', ('all', 'y', ('<=', X('x'), X('y'))))),
]


def run_suite(d, w, verbose=True):
    kernel = Kernel(TAG, {})
    store = {}
    S = Search(kernel, store, d, w)
    rows, solved = [], 0
    for name, phi in TARGETS:
        t0 = time.perf_counter()
        S.nodes = 0
        C = S.prove(phi)
        dt = time.perf_counter() - t0
        verdict, ok = "", False
        if C is not None:
            try:
                kernel.check(store, [], phi, C)
                ok, verdict = True, "ok"
                store[name] = phi
            except Reject as e:
                verdict = "REJECTED: %s" % e
        solved += ok
        rows.append(dict(target=name, tier=tier_name(phi),
                         formula=show(phi), proved=C is not None,
                         certified=ok, verdict=verdict,
                         nodes=S.nodes, seconds=round(dt, 4)))
        if verbose:
            print(" %-12s %-7s %-34s %s"
                  % (name, tier_name(phi), show(phi)[:34],
```

```
                    ("CERT ok, %d nodes" % S.nodes) if ok else
                    (verdict or "not proved")))
    return solved, rows, kernel


def tier_profile(rows):
    prof = defaultdict(lambda: [0, 0])
    for r in rows:
        prof[r["tier"]][1] += 1
        if r["certified"]:
            prof[r["tier"]][0] += 1
    return {k: dict(certified=v[0], targets=v[1])
            for k, v in sorted(prof.items())}


# ============================================================================
def cmd_run(a):
    print("=" * 74)
    print(" ASCENT d=%d (depth) w=%d (witness) r=%d (reflect)"
          % (a.depth, a.witness, a.reflect))
    print("=" * 74)
    print("\n arithmetic targets\n")
    solved, rows, kernel = run_suite(a.depth, a.witness)

    print("\n reflection ladder\n")
    names = {}
    store2 = {}
    K2 = Kernel(TAG, names)
    level, rungs = climb(K2, store2, names, a.reflect)

    prof = tier_profile(rows)
    highest = max([r["tier"] for r in rows if r["certified"]],
                  default="none", key=lambda s: (s != "none", s))
    tiers_hit = sorted({r["tier"] for r in rows if r["certified"]})

    print("\n" + "-" * 74)
    print(" certified %d/%d targets | tiers reached: %s"
          % (solved, len(rows), ", ".join(tiers_hit) or "none"))
    print(" lev(A) = %d (each rung kernel-certified; r is the only cap)"
          % level)
    print(" kernel calls: %d suite + %d ladder"
          % (kernel.calls, K2.calls))
    print("-" * 74)

    os.makedirs(a.out, exist_ok=True)
    with open(os.path.join(a.out, "ascent_d%d_w%d_r%d.json"
                           % (a.depth, a.witness, a.reflect)), "w") as f:
        json.dump(dict(params=dict(d=a.depth, w=a.witness, r=a.reflect),
                       certified=solved, targets=len(rows),
                       tier_profile=prof, tiers_reached=tiers_hit,
                       lev=level, rows=rows, rungs=rungs), f, indent=2)
    return 0


def cmd_grid(a):
    """Proposition 2, empirically: sweep the knobs and show what moves."""
    print("=" * 74)
    print(" PARAMETER SWEEP -- what each knob buys")
    print("=" * 74)
    print("\n %-4s %-4s %-4s %-10s %-8s %-22s %s"
          % ("d", "w", "r", "certified", "lev", "tiers reached", "nodes"))
    print(" " + "-" * 70)
    out = []
    for d in a.depths:
        for w in a.witnesses:
            for r in a.reflects:
                solved, rows, kernel = run_suite(d, w, verbose=False)
                names, store2 = {}, {}
                K2 = Kernel(TAG, names)
                level, _ = climb(K2, store2, names, r, verbose=False)
                tiers = sorted({x["tier"] for x in rows if x["certified"]})
                nodes = sum(x["nodes"] for x in rows)
                print(" %-4d %-4d %-4d %-10s %-8d %-22s %s"
```

```
                    % (d, w, r, "%d/%d" % (solved, len(rows)), level,
                       ",".join(tiers) or "-", f"{nodes:,}"))
            out.append(dict(d=d, w=w, r=r, certified=solved,
                            targets=len(rows), lev=level,
                            tiers=tiers, nodes=nodes))
    os.makedirs(a.out, exist_ok=True)
    with open(os.path.join(a.out, "ascent_grid.json"), "w") as f:
        json.dump(out, f, indent=2)

    lev_by_r = defaultdict(set)
    cert_by_dw = defaultdict(set)
    for o in out:
        lev_by_r[o["r"]].add(o["lev"])
        cert_by_dw[(o["d"], o["w"])].add(o["certified"])
    ind_r = all(len(v) == 1 for v in lev_by_r.values())
    ind_dw = all(len(v) == 1 for v in cert_by_dw.values())
    print("\n lev depends only on r : %s" % ("YES" if ind_r else "NO"))
    print(" certified count depends only on (d,w) : %s"
          % ("YES" if ind_dw else "NO"))
    print(" -> Proposition 2 holds on this grid" if (ind_r and ind_dw)
          else " -> Proposition 2 FAILS on this grid")
    return 0


def cmd_soundness(a):
    """Hand the kernel certificates that must be rejected."""
    print("=" * 74)
    print(" KERNEL SOUNDNESS PROBES -- each of these MUST be rejected")
    print("=" * 74 + "\n")
    names = {"<f>": ('=', ('n', 0), ('n', 1))}
    K = Kernel(TAG, names)
    store = {}
    bad = [
        ("false by calc", ('=', ('n', 0), ('n', 1)), ('calc',)),
        ("lemma that is not stored",
         ('=', ('n', 0), ('n', 1)), ('lemma', 'nope')),
        ("nec for an unproved formula",
         ('pr', TAG, "<f>"), ('nec', ('calc',))),
        ("nec with the wrong tag",
         ('pr', "<Other>", "<f>"), ('nec', ('calc',))),
        ("gen from a single instance",
         ('all', 'x', ('=', X('x'), ('n', 0))),
         ('gen', 'z', ('calc',))),
        ("gen with a captured eigenvariable",
         ('all', 'x', ('=', X('x'), ('c', 'k'))),
         ('gen', 'k', ('calc',))),
        ("wit with no witness",
         ('ex', 'x', ('<', ('s', X('x')), X('x'))), ('wit', ('n', 3),
                                                     ('calc',))),
        ("induction without a base",
         ('all', 'x', ('<', X('x'), ('n', 0))),
         ('ind', ('calc',), ('calc',))),
    ]
    caught = 0
    for label, phi, C in bad:
        try:
            K.check(store, [], phi, C)
            print(" %-34s ACCEPTED <-- UNSOUND" % label)
        except Reject as e:
            caught += 1
            print(" %-34s rejected (%s)" % (label, str(e)[:30]))
    print("\n %d/%d rejected%s" % (caught, len(bad),
                                  "" if caught == len(bad)
                                  else " <-- KERNEL IS UNSOUND"))
    return 0 if caught == len(bad) else 1


def main():
    ap = argparse.ArgumentParser(
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("-d", "--depth", type=int, default=3)
    ap.add_argument("-w", "--witness", type=int, default=10)
```

```
    ap.add_argument("-r", "--reflect", type=int, default=4)
    ap.add_argument("--out", default="results_ascent")
    ap.add_argument("--grid", action="store_true")
    ap.add_argument("--soundness", action="store_true")
    ap.add_argument("--depths", type=int, nargs="+", default=[1, 2, 3, 4])
    ap.add_argument("--witnesses", type=int, nargs="+", default=[1, 5, 20])
    ap.add_argument("--reflects", type=int, nargs="+", default=[1, 3, 8])
    a = ap.parse_args()
    if a.soundness:
        return cmd_soundness(a)
    if a.grid:
        return cmd_grid(a)
    return cmd_run(a)


if __name__ == "__main__":
    sys.exit(main())
```